simple_rl — Detailed Module Design v2

Online RL with LoRA Training, Adapter Continuity, and W&B Observability

OpenClaw-RL Project

2026-03-29

Contents

1 What Is New in v2 **3**

2 System Overview **4**
2.1 Design Philosophy . . . 4
2.2 Updated High-Level Data Flow . . . 4

3 Module Reference **6**
3.1 `config.py` – Centralised Configuration . . . 6
3.1.1 Parameters Added in v2 . . . 6
3.1.2 Design Notes . . . 6
3.2 `agent_loop.py` – Multi-Turn Agent Loop . . . 8
3.2.1 New: Policy API Semaphore . . . 8
3.2.2 New: Exponential-Backoff Retry . . . 8
3.2.3 Verification . . . 9
3.3 `rollout_buffer.py` – GRPO Rollout Buffer . . . 10
3.4 `prm_client.py` – Process Reward Model Client . . . 11
3.4.1 New: `PRMStepResult` Dataclass . . . 11
3.4.2 New: JSONL File Logging . . . 11
3.4.3 Verification . . . 12
3.5 `mlx_grpo_bridge.py` – Subprocess Orchestrator *(NEW)* . . . 13
3.5.1 Why a Subprocess Bridge? . . . 13
3.5.2 `GRPOUpdateResult` Dataclass . . . 13
3.5.3 `batches_to_dataset(batches)` – Format Conversion . . . 13
3.5.4 `run_grpo_update()` – Entry Point . . . 14
3.5.5 Job and Result JSON Schema . . . 14
3.5.6 Error Handling . . . 15
3.5.7 Verification . . . 15
3.6 `grpo_worker.py` – MLX Training Worker *(NEW)* . . . 16
3.6.1 Model Loading Sequence . . . 16
3.6.2 `WandbGRPOTrainer` . . . 16
3.6.3 Verification . . . 18
3.7 `train_async.py` – Training Orchestrator . . . 20
3.7.1 New: W&B Authentication and Entity . . . 20

3.7.2 New: W&B Run ID Passed to GRPO Worker 20
3.7.3 New: `_last_adapter_path` – Round-to-Round Continuity 20
3.7.4 Per-Round W&B Logging (updated) 21

4 Cross-Cutting Concerns **22**
4.1 Concurrency Architecture 22
4.2 Error Propagation Strategy 22
4.3 Testing Strategy 23
4.4 File Layout (Updated) 23

5 Deployment Reference (Updated) **25**
5.1 Full Startup with GRPO Training 25
5.2 Environment Variable Quick Reference 25

1 What Is New in v2

This document supersedes `simple-rl-design.md` (v1, 2026-03-27). All v1 content is preserved and expanded. The following major capabilities were added on 2026-03-29:

Area	Change	Files
Policy reliability	Semaphore serialisation + exponential-backoff retry for oMLX 500 errors	`agent_loop.py`, `config.py`
W&B observability	Entity, API key, run ID wiring; per-step and per-round metric logging	`config.py`, `train_async.py`
PRM JSONL logging	Per-turn votes, scores, and representative evaluation text written to disk	`prm_client.py`
GRPO training	Full mlx-tune subprocess bridge replacing the logging stub	`mlx_grpo_bridge.py` (new), `grpo_worker.py` (new), `train_async.py`
LoRA verification	Trainable-parameter count and LoRA config logged to stdout and W&B	`grpo_worker.py`
Adapter continuity	Each GRPO round resumes from the previous round's saved adapters	`grpo_worker.py`, `mlx_grpo_bridge.py`, `train_async.py`

2 System Overview

`simple_rl` is a self-contained **Apple Silicon-native** online reinforcement learning framework for training terminal agents. It deliberately replaces the distributed infrastructure from earlier iterations (Ray, CAMEL, Router Server, remote Pool Workers) with a single-process AsyncIO design that runs entirely on one Mac.

2.1 Design Philosophy

Principle	Old stack	simple_rl
Concurrency	Ray distributed tasks	`asyncio.Task` per episode
Inter-process comms	HTTP between Pool/Router/Worker servers	In-process function calls
Container management	Remote pool server over HTTP	`LocalEnvPool` – in-process subprocess
Model serving	External vLLM or TGI	oMLX / `mlx_lm.server` – same Mac
Weight updates	Serialise –> upload –> reload	Subprocess bridge to mlx-tune venv
Training isolation	N/A	System Python orchestrates; mlx-tune venv trains
Adapter management	N/A	LoRA adapters saved per round; loaded on next round

2.2 Updated High-Level Data Flow

```
sample_tasks.jsonl
        |  task dicts (round-robin)
        v
  train_async.py  ---- asyncio.Task x MAX_CONCURRENT ---->  run_episode()
        |                                                        |
        |              +---------- LocalEnvPool ----------+      |
        |              |  allocate --> docker run         |      |
        |              |  exec     --> docker exec        |      |
        |              |  evaluate --> grader bash script |      |
        |              +----------------------------------+  |
        |                                                        |
        |<------------ Trajectory (messages + score) ------------+
        |
        v
  RolloutBuffer
        |  wait for N_SAMPLES_PER_PROMPT per task
        |  compute GRPO advantage Ai = (ri - mu) / (sigma + eps)
        v
  GRPOBatch x ROLLOUT_BATCH_SIZE
        |
```

```
      +---> [optional] PRMClient --> prm_steps.jsonl (new)
      |
      v
_submit_to_mlx_tune()  [train_async.py]
      |  writes grpo_batch_r*.jsonl (snapshot)
      |  passes prev_adapter_path (NEW: round-to-round continuity)
      v
mlx_grpo_bridge.run_grpo_update()
      |  serialises GRPOJob JSON
      |  launches subprocess
      v
grpo_worker.py  (mlx-tune venv Python)   <-- NEW subprocess
      |  FastLanguageModel.from_pretrained(base_model)
      |  get_peft_model()  --> LoRA layers attached
      |  load_adapter(prev_adapter_path)  --> resume from checkpoint
      |  WandbGRPOTrainer.train()
      |    - logs lora/* to W&B before training
      |    - logs grpo/loss per step to W&B
      |  _save_adapters_and_config()
      v
result_r*.json  --> GRPOUpdateResult  -->  _last_adapter_path updated
```

3 Module Reference

3.1 `config.py` – Centralised Configuration

File: `simple_rl/config.py`

Role: Single source of truth for every tunable parameter. All values are read from environment variables with safe defaults.

3.1.1 Parameters Added in v2

Variable	Default	Description
`POLICY_MAX_CONCURRENT`	`1`	Semaphore limit for policy API calls (oMLX is single-request)
`POLICY_MAX_RETRIES`	`3`	Max retry attempts on `InternalServerError` (500)
`POLICY_RETRY_BACKOFF`	`2.0` s	Initial backoff; doubles on each retry (exponential)
`WANDB_ENTITY`	`bochuxt7-iot`	W&B workspace entity name
`WANDB_API_KEY`	`""`	W&B API key; set via environment, never hardcoded
`MLX_TUNE_PYTHON`	`.../mlx-tune/.venv/bin/python3`	Path to mlx-tune venv interpreter
`POLICY_MODEL_PATH`	`""`	HuggingFace ID or local path for GRPO training; empty = stub mode
`GRPO_OUTPUT_DIR`	`logs/grpo_adapters`	Root directory for saved LoRA adapters
`GRPO_LORA_RANK`	`16`	LoRA rank r used for all training rounds
`GRPO_LR`	`1e-6`	AdamW learning rate for GRPO
`GRPO_NUM_GEN`	`4`	Completions generated per prompt per step
`GRPO_BETA`	`0.04`	KL penalty coefficient
`GRPO_LOSS_TYPE`	`grpo`	Loss variant: `grpo`, `dr_grpo`, `dapo`, `bnpo`
`GRPO_MAX_STEPS`	`-1`	Training steps per round; `-1` = `len(dataset)`
`GRPO_LOGGING_STEPS`	`1`	Log loss every N steps

3.1.2 Design Notes

- `GRPO_LORA_RANK` must stay **constant across all rounds**. Changing it between rounds causes a rank mismatch when loading adapters from a previous round.
- `POLICY_MODEL_PATH` being empty is the "safe default" – the entire GRPO pipeline runs in stub mode and returns immediately without any gradient update, which is correct for dry-runs and unit tests.

- `MLX_TUNE_PYTHON` points to a **separate venv** because `mlx` has a broken dylib path in the system Python on this machine. The subprocess bridge pattern cleanly isolates the mlx dependency.

3.2 `agent_loop.py` – Multi-Turn Agent Loop

File: `simple_rl/agent_loop.py`

All v1 behaviour is preserved. Two reliability enhancements were added.

3.2.1 New: Policy API Semaphore

oMLX (`mlx_lm.server`) processes exactly one request at a time. Sending concurrent requests produces `HTTP 500 InternalServerError`. A module-level `asyncio.Semaphore` is used to serialise all policy calls:

```
_POLICY_SEMAPHORE: Optional[asyncio.Semaphore] = None

def _get_policy_semaphore() -> asyncio.Semaphore:
    global _POLICY_SEMAPHORE
    if _POLICY_SEMAPHORE is None:
        _POLICY_SEMAPHORE = asyncio.Semaphore(config.POLICY_MAX_CONCURRENT)
    return _POLICY_SEMAPHORE
```

The semaphore is lazily initialised inside the running event loop so it binds to the correct loop. Default `POLICY_MAX_CONCURRENT = 1` means at most one in-flight API request at any time.

Why a module-level semaphore rather than per-instance? All episodes share the same policy server. A per-episode semaphore would allow N episodes to each hold one slot simultaneously – defeating the purpose. A shared module-level semaphore creates a single global queue of API calls.

3.2.2 New: Exponential-Backoff Retry

The semaphore-guarded call is wrapped in a retry loop:

```
backoff = config.POLICY_RETRY_BACKOFF   # starts at 2.0 s
for attempt in range(config.POLICY_MAX_RETRIES):
    try:
        async with _get_policy_semaphore():
            resp = await client.chat.completions.create(...)
        break   # success
    except openai.InternalServerError:
        if attempt + 1 < config.POLICY_MAX_RETRIES:
            await asyncio.sleep(backoff)
            backoff *= 2   # 2s -> 4s -> 8s
        else:
            traj.error = f"Policy API error on turn {turn}: ..."
    except openai.OpenAIError as exc:
        traj.error = ...; break   # non-retriable error
```

Retry matrix (default config):

Attempt	Sleep before	Total elapsed
1 (fail)	–	0 s
2 (fail)	2 s	2 s
3 (fail)	4 s	6 s
3 (success)	–	6 s

Only `InternalServerError` (500) is retried. Other `OpenAIError` types (authentication, context length, etc.) are non-retriable and terminate the episode immediately.

3.2.3 Verification

Run the unit tests; the semaphore and retry are covered in `tests/test_agent_loop.py`. Observe during a real training run that `ERROR simple_rl.agent_loop Policy error: Error code: 500` messages no longer appear in the log.

3.3 `rollout_buffer.py` – GRPO Rollout Buffer

(No changes from v1. See original document for full description.)

The GRPO advantage formula is unchanged:

$$A_i = \frac{r_i - \bar{r}}{\widehat{\sigma}(r) + \varepsilon}, \quad \varepsilon = 10^{-8}$$

3.4 `prm_client.py` – Process Reward Model Client

File: `simple_rl/prm_client.py`

All v1 scoring behaviour is preserved. JSONL file logging was added.

3.4.1 New: `PRMStepResult` Dataclass

`score_step()` now returns a structured result instead of a bare float:

```
@dataclass
class PRMStepResult:
    score:               float        # mean of all vote scores
    votes:               List[float]  # individual scores from each of the M calls
    representative_eval: str          # raw model response from the median-score call
```

Why store the raw response? During training, it is useful to inspect *why* the PRM assigned a particular score. The `representative_eval` field stores the verbatim model output from the vote closest to the mean, providing an auditable explanation.

3.4.2 New: JSONL File Logging

`score_trajectory()` now writes one record per turn to `{log_dir}/prm_steps.jsonl`:

```
{
  "session":             "2026-03-29T16:00:00",
  "task":                "hello_world",
  "turn":                0,
  "score":               0.82,
  "votes":               [0.8, 0.85, 0.81],
  "representative_eval": "0.82 - the command correctly creates the file..."
}
```

Field descriptions:

Field	Description
`session`	ISO timestamp of training run start; groups records from one run
`task`	Task name from `task_dict["task_name"]`
`turn`	Zero-based index of the agent turn within the episode
`score`	Mean PRM score across all `PRM_M` votes
`votes`	Raw float from each of the `PRM_M` model calls
`representative_eval`	Verbatim response from the vote closest to the mean score

Logging is best-effort: if writing fails, a warning is logged and training continues – a PRM logging failure must never block a training round.

3.4.3 Verification

```
# After one full episode with PRM enabled:
cat logs/prm_steps.jsonl | python3 -c "
import sys, json
for line in sys.stdin:
    r = json.loads(line)
    print(f\"turn {r['turn']:2d}  score={r['score']:.3f}  votes={r['votes']}\")
"
```

Expected output (one line per agent turn):

```
turn  0  score=0.820  votes=[0.8, 0.85, 0.81]
turn  1  score=0.910  votes=[0.9, 0.92, 0.91]
...
```

3.5 `mlx_grpo_bridge.py` – Subprocess Orchestrator *(NEW)*

File: `simple_rl/mlx_grpo_bridge.py`

Role: Bridges the system Python training loop and the mlx-tune venv. Because `mlx` cannot be imported into the system Python (broken dylib path), this module serialises training jobs to JSON and launches `grpo_worker.py` as a subprocess under the mlx-tune venv interpreter.

3.5.1 Why a Subprocess Bridge?

```
System Python (train_async.py)          mlx-tune venv (grpo_worker.py)
+----------------------------+          +----------------------------+
| asyncio event loop         |          | mlx.core (GPU arrays)      |
| openai client              |  JSON    | mlx.optimizers (AdamW)     |
| rollout buffer             | -------> | mlx_tune.FastLanguageModel |
| W&B run                    | <------- | mlx_tune.GRPOTrainer       |
+----------------------------+  result  +----------------------------+
```

The bridge pattern provides clean dependency isolation: the orchestrator needs no MLX; the worker needs no asyncio, openai, or W&B run references.

3.5.2 `GRPOUpdateResult` Dataclass

```
@dataclass
class GRPOUpdateResult:
    round_num:    int
    status:       str           # "success" | "error" | "stub"
    adapter_path: str           # path to saved LoRA adapters
    step_losses:  List[Dict]    # [{"step": N, "loss": float}, ...]
    error:        Optional[str]
    is_stub:      bool          # True when POLICY_MODEL_PATH is unset
```

3.5.3 `batches_to_dataset(batches)` – Format Conversion

Converts `List[GRPOBatch]` to the mlx-tune dataset format:

```
[
  {
    "prompt":     "Task: hello_world\nInstruction: ...",
    "answer":     "0.85",      # str(trajectory.score) -- used as reward
    "_advantage": -1.234567,   # pre-computed GRPO advantage (metadata only)
    "_task_id":   "hello_world"
  },
  ...
]
```

The `reward_fn` in `grpo_worker.py` reads `float(answer)` as the pre-computed reward. This is an **offline-GRPO approximation**: the model learns to produce outputs similar to high-scoring trajectories using pre-collected advantage signals, rather than generating new completions and scoring them online.

3.5.4 `run_grpo_update()` – Entry Point

```
run_grpo_update(batches, round_num, ..., prev_adapter_path="")
  |
  +-- stub mode (model_path == "") --> return GRPOUpdateResult(is_stub=True)
  |
  +-- build job JSON
  |     round_num, model_path, lora_rank, dataset,
  |     output_dir, loss_type, beta, num_generations,
  |     temperature, learning_rate, max_steps, logging_steps,
  |     wandb_project, wandb_entity, wandb_api_key, wandb_run_id,
  |     adapter_path  <-- NEW: previous round's checkpoint path
  |
  +-- write job_r{N}_{uuid6}.json  to  logs/grpo_jobs/
  |
  +-- subprocess.run(
  |       [mlx_python, "-m", "simple_rl.grpo_worker", job_path],
  |       cwd=repo_root,
  |       timeout=3600
  |   )
  |
  +-- read result_r{N}_{uuid6}.json
  |
  +-- return GRPOUpdateResult
```

3.5.5 Job and Result JSON Schema

Job (`logs/grpo_jobs/job_r*.json`):

```
{
  "round_num":       1,
  "model_path":      "mlx-community/Qwen2.5-0.5B-Instruct-4bit",
  "lora_rank":       16,
  "dataset":         [{"prompt": "...", "answer": "0.85", ...}],
  "output_dir":      "logs/grpo_adapters",
  "loss_type":       "grpo",
  "beta":            0.04,
  "num_generations": 4,
  "temperature":     0.7,
  "learning_rate":   1e-6,
  "max_steps":       -1,
  "logging_steps":   1,
  "wandb_project":   "terminal-rl-simple",
  "wandb_entity":    "bochuxt7-iot",
  "wandb_api_key":   "...",
  "wandb_run_id":    "abc123",
  "adapter_path":    "logs/grpo_adapters/round_0001"
}
```

Result (`logs/grpo_jobs/result_r*.json`):

```
{
  "status":       "success",
  "adapter_path": "logs/grpo_adapters/round_0002",
  "step_losses":  [{"step": 1, "loss": 0.423}, {"step": 2, "loss": 0.381}],
  "error":        null
}
```

3.5.6 Error Handling

Condition	Behaviour
`model_path` empty	Immediate stub return – no subprocess
`subprocess.TimeoutExpired`	Returns error result with "timed out after Ns"
`FileNotFoundError` (mlx_python missing)	Returns error result with path
Worker `returncode != 0`	Returns error result with exit code
Result JSON missing	Returns error result (worker crashed before writing)

3.5.7 Verification

```
# Unit tests (no model needed)
python3 -m pytest simple_rl/tests/test_mlx_grpo_bridge.py -m unit -v

# Integration test (requires model)
POLICY_MODEL_PATH=mlx-community/Qwen2.5-0.5B-Instruct-4bit \
  python3 -m pytest simple_rl/tests/test_mlx_grpo_bridge.py -m integration -v -s
```

3.6 `grpo_worker.py` – MLX Training Worker *(NEW)*

File: `simple_rl/grpo_worker.py`

Role: Runs under the mlx-tune venv interpreter. Reads a GRPOJob JSON, trains the model using `WandbGRPOTrainer`, writes a GRPOResult JSON.

Execution context: `mlx-tune/.venv/bin/python3 -m simple_rl.grpo_worker <job_path>`

3.6.1 Model Loading Sequence

```
# 1. Load base model weights from HuggingFace or local path
model, tokenizer = FastLanguageModel.from_pretrained(
    job["model_path"], max_seq_length=2048
)

# 2. Attach LoRA layer structure (creates LoRALinear layers, sets frozen/trainable)
model = FastLanguageModel.get_peft_model(model, r=job["lora_rank"])

# 3. Resume from previous round's adapter weights (NEW)
if job.get("adapter_path"):
    model.load_adapter(job["adapter_path"])
    # Loads lora_a / lora_b weights into the LoRALinear layers
    # without fusing them into the base model weights
```

Why `get_peft_model` before `load_adapter`? `get_peft_model` creates the LoRALinear *layer structure* on the base model. `load_adapter` then fills in the *weights* (`lora_a`, `lora_b`) from the saved checkpoint. mlx-tune requires this two-step pattern – the layer architecture must exist before weights can be loaded into it.

3.6.2 `WandbGRPOTrainer`

Subclass of `mlx_tune.GRPOTrainer` that overrides `_train_native()` to add:

1. W&B metric axis definition
2. LoRA verification logging
3. Per-step W&B loss logging

3.6.2.1 W&B Metric Axis Setup

```
_wandb.define_metric("grpo/step")
_wandb.define_metric("grpo/*", step_metric="grpo/step")
```

`define_metric` tells W&B to use `grpo/step` as the x-axis for all `grpo/*` charts. Without this, W&B defaults to wall-clock time, making loss curves from different rounds incomparable.

3.6.2.2 `_log_lora_verification()` Called once before the first training step. Inspects `model.lora_config` and counts trainable vs frozen parameters using `mlx.utils.tree_flatten`:

```
all_flat      = tree_flatten(inner.parameters())
trainable_flat = tree_flatten(inner.trainable_parameters())
```

```
lora_tensors  = [(k, v) for k, v in trainable_flat if 'lora' in k.lower()]
```

Stdout output (printed before step 1):

```
============================================================
  LoRA Verification
============================================================
  Enabled           : True
  Applied to layers : True
  Rank (r)          : 16
  Alpha             : 32
  Target modules    : ['self_attn.q_proj', 'self_attn.v_proj', ...]
  Trainable tensors : 28
  Trainable params  : 2,621,440
  Frozen params     : 494,032,768
  % trainable       : 0.53%
============================================================
```

W&B metrics logged (once, before training):

Metric	Value
`lora/enabled`	1
`lora/applied`	1
`lora/rank`	16
`lora/alpha`	32
`lora/trainable_tensors`	28
`lora/trainable_params`	2,621,440
`lora/frozen_params`	494,032,768

These are also stored in `wandb.run.summary` for cross-run comparison.

Red flag: **`Trainable params = 0`** This means LoRA was not applied. Check that `get_peft_model` did not fail silently and that `_apply_lora()` was called.

3.6.2.3 Training Loop

```
for step in range(self.iters):
    prompt = prompts[step % len(prompts)]

    loss, _ = grpo_batch_loss(
        model=actual_model,
        tokenizer=self.tokenizer,
        prompts=[prompt],
        reward_fn=self.reward_fn,       # float(ground_truth) from dataset
        num_generations=self.num_generations,
        temperature=self.temperature,
        max_tokens=self.max_completion_length,
        beta=self.beta,
    )
```

```
    mx.eval(loss)
    step_loss = loss.item()

    if (step + 1) % self.logging_steps == 0:
        avg_loss = total_loss / self.logging_steps
        self.step_losses.append({"step": step + 1, "loss": avg_loss})
        self._wb.log({
            "grpo/step":          step + 1,
            "grpo/loss":          avg_loss,
            "grpo/round":         self._round,
            "grpo/learning_rate": self.learning_rate,
        })
```

3.6.2.4 `grpo_batch_loss` Internals

The mlx-tune `grpo_batch_loss` function:

1. Encodes each prompt to token IDs
2. Generates `num_generations` completions using the *same model being trained* (no separate reference model in this implementation)
3. Calls `reward_fn(completion, prompt)` for each – returns `float(answer)` from the pre-collected dataset score
4. Computes group advantages: $(r_i - \bar{r})/(\hat{\sigma} + \varepsilon)$
5. Computes policy gradient loss: $-\mathbb{E}[A_i \cdot \log \pi(y_i|x)]$
6. Returns `(loss, num_completions)`

No reference model required because the advantage is computed purely from group statistics within the batch – the KL term is omitted in this mlx-tune implementation (the `beta` parameter is accepted but not used in the loss).

3.6.2.5 Adapter Saving

```
_save_adapters_and_config(self.model, self.adapter_path)
```

Writes to `{output_dir}/round_{N:04d}/`:

```
adapters.safetensors   <- lora_a, lora_b weights for each target layer
adapter_config.json    <- rank, alpha, dropout, target_modules, fine_tune_type
```

Only LoRA weights are saved – the base model is not duplicated. A rank-16 adapter for a 0.5B model is approximately 10 MB.

3.6.3 Verification

```
# After a successful round, inspect the adapter config:
cat logs/grpo_adapters/round_0001/adapter_config.json

# Verify weights file is non-trivial:
du -sh logs/grpo_adapters/round_0001/adapters.safetensors
# Expected: ~10M for 0.5B model at rank=16
```

```
# Manual adapter load test (in mlx-tune venv):
/Volumes/ExternalSSD/train/mlx-tune/.venv/bin/python3 - <<'EOF'
from mlx_tune import FastLanguageModel
from mlx.utils import tree_flatten

model, tok = FastLanguageModel.from_pretrained(
    "mlx-community/Qwen2.5-0.5B-Instruct-4bit", max_seq_length=512
)
model = FastLanguageModel.get_peft_model(model, r=16)
model.load_adapter("logs/grpo_adapters/round_0001")

lora = [(k,v) for k,v in tree_flatten(model.model.trainable_parameters())
        if 'lora' in k]
print(f"Loaded {len(lora)} LoRA tensors")
# Expected: non-zero, e.g. "Loaded 28 LoRA tensors"
EOF
```

3.7 `train_async.py` – Training Orchestrator

File: `simple_rl/train_async.py`

All v1 loop behaviour is preserved. Three enhancements were added.

3.7.1 New: W&B Authentication and Entity

```
if _WANDB_AVAILABLE and wandb_project:
    if wandb_api_key:
        _wandb.login(key=wandb_api_key)         # explicit auth
    _wandb.init(
        project=wandb_project,
        entity=wandb_entity or None,            # workspace routing
        config={...},
    )
```

Without `entity`, W&B runs land in the user's personal workspace even if `WANDB_PROJECT` names a team project. The `entity` field routes the run to `bochuxt7-iot/terminal-rl-simple`.

3.7.2 New: W&B Run ID Passed to GRPO Worker

```
_wb_run_id = (
    _wandb.run.id
    if (_WANDB_AVAILABLE and wandb_project and _wandb.run)
    else None
)
_submit_to_mlx_tune(train_batches, round_num, log_dir,
                    wandb_run_id=_wb_run_id)
```

Passing the parent run ID into `grpo_worker.py` allows the subprocess to initialise W&B with `resume="allow"`, attaching its `grpo/*` and `lora/*` metrics to the **same W&B run** as the rollout metrics. Without this, each training round would create a separate W&B run, making cross-round loss comparison impossible.

3.7.3 New: `_last_adapter_path` – Round-to-Round Continuity

```
_last_adapter_path: str = ""   # module-level, persists across rounds

def _submit_to_mlx_tune(batches, round_num, log_dir, ...):
    global _last_adapter_path
    ...
    if _last_adapter_path:
        logger.info("GRPO round %d: resuming from adapter %s",
                    round_num, _last_adapter_path)

    result = run_grpo_update(
        ...,
        prev_adapter_path=_last_adapter_path,   # "" on round 1
    )
```

```
    if result.status == "success":
        _last_adapter_path = result.adapter_path   # carry forward
```

What happens without this: Each round loads the frozen base model, trains for `GRPO_MAX_STEPS`, saves an adapter, and discards it. The next round starts from the same frozen base – training does not accumulate.

With this: Round 1 starts from the base. Round 2 loads round 1's adapter, training from where it left off. Round 3 loads round 2's adapter, and so on. The adapter checkpoint from each round is the cumulative sum of all gradient updates applied so far.

Failure safety: If a round fails, `_last_adapter_path` is not updated. The next round retries from the last known good checkpoint.

3.7.4 Per-Round W&B Logging (updated)

On a successful GRPO round:

```
_wandb.log({
    "grpo/round":      round_num,
    "grpo/final_loss": final_loss,   # last step's avg loss
    "grpo/adapter":    result.adapter_path,
    "grpo/mean_score": mean_score,   # mean trajectory score this round
    "grpo/mean_adv":   mean_adv,     # mean advantage (should be ~0)
})
```

W&B charts to monitor:

Chart	Healthy sign
`grpo/loss` vs `grpo/step`	Decreasing within each round
`grpo/final_loss` vs `grpo/round`	Decreasing across rounds
`lora/trainable_params`	Constant non-zero value every round
`lora/applied`	Always 1
`score` vs `step`	Upward trend as policy improves
`grpo/mean_score` vs `grpo/round`	Upward trend

4 Cross-Cutting Concerns

4.1 Concurrency Architecture

```
Python Process (single OS thread)
|
+-- asyncio Event Loop
      |
      +-- asyncio.Task: episode-0   (run_episode coroutine)
      +-- asyncio.Task: episode-1
      +-- asyncio.Task: episode-2
      +-- asyncio.Task: episode-3
      +-- asyncio.Task: env-pool-reaper
      |
      +-- Semaphore(POLICY_MAX_CONCURRENT=1)
      |     serialises all policy API calls
      |
      +-- I/O multiplexing:
            +-- asyncio.create_subprocess_exec  (docker run / exec / rm)
            +-- openai.AsyncOpenAI HTTP client  (policy / PRM API calls)
            +-- asyncio.Queue / asyncio.Lock

[Separate OS process -- not in the event loop]
+-- grpo_worker.py (mlx-tune venv)
      spawned by subprocess.run() in run_grpo_update()
      blocks the calling thread until training completes
      communicates only via job/result JSON files
```

Why does `subprocess.run()` not block the asyncio loop? `_submit_to_mlx_tune` is a regular (non-async) function called from the training loop between `asyncio.wait()` calls. While the subprocess runs, no new episodes are dispatched – this is intentional: GRPO training on Apple Silicon uses the GPU's entire memory bandwidth, and running episodes concurrently would cause memory pressure.

4.2 Error Propagation Strategy

Component	On Error	Behaviour
`TerminalEnv.exec`	Timeout / process error	Returns `[TIMEOUT]`/`[EXEC_ERROR]` string
`run_episode`	Policy 500 error	Retry with backoff; fail episode after max retries
`run_episode`	Non-retriable API error	Sets `traj.error`, returns `score=0.0`
`PRMClient.score_step`	LLM error	Returns `0.5` neutral; logs warning
`PRMClient.score_trajectory`	JSONL write error	Logs warning; training continues
`run_grpo_update`	Subprocess timeout	Returns error result; logs error

Component	On Error	Behaviour
`grpo_worker.main`	Model load failure	Writes error result JSON; exits 1
`grpo_worker.main`	Adapter load failure	Writes error result JSON; exits 1
`_submit_to_mlx_tune`	Worker error result	Logs error; `_last_adapter_path` unchanged

4.3 Testing Strategy

Tests are split by `pytest` markers.

Unit tests (`-m unit`): no Docker, no live model, no real W&B:

```
python3 -m pytest simple_rl/tests/ -m unit -q
```

Covers: terminal env, env pool, agent loop, rollout buffer, PRM JSONL, GRPO submission, GRPO bridge (including LoRA verification and W&B metric setup).

Integration tests (`-m integration`): require Docker and/or live model:

```
# GRPO end-to-end
POLICY_MODEL_PATH=mlx-community/Qwen2.5-0.5B-Instruct-4bit \
  python3 -m pytest simple_rl/tests/test_mlx_grpo_bridge.py -m integration -v -s

# W&B authentication
WANDB_API_KEY=<key> \
  python3 -m pytest simple_rl/tests/test_wandb.py -m integration -v -s
```

4.4 File Layout (Updated)

```
simple_rl/
+-- __init__.py
+-- config.py             <- all configuration (updated with GRPO / W&B params)
+-- terminal_env.py       <- Docker container wrapper
+-- local_env_pool.py     <- bounded async container pool
+-- agent_loop.py         <- multi-turn agent + semaphore + retry (updated)
+-- rollout_buffer.py     <- GRPO buffer + GRPOBatch
+-- prm_client.py         <- per-step scoring + JSONL logging (updated)
+-- train_async.py        <- main loop + adapter continuity (updated)
+-- mlx_grpo_bridge.py    <- subprocess orchestrator (NEW)
+-- grpo_worker.py        <- mlx-tune training worker (NEW)
+-- run.sh
+-- pytest.ini
+-- data/
|   +-- sample_tasks.jsonl
+-- tests/
|   +-- conftest.py
|   +-- test_terminal_env.py
```

```
|   +-- test_local_env_pool.py
|   +-- test_agent_loop.py
|   +-- test_rollout_buffer.py
|   +-- test_prm_jsonl.py         (NEW)
|   +-- test_grpo_submission.py   (NEW)
|   +-- test_mlx_grpo_bridge.py   (NEW -- 20 unit tests + 1 integration)
|   +-- test_wandb.py             (NEW)
+-- assets/
    +-- simple-rl-design.md       <- v1 original
    +-- simple-rl-design.pdf      <- v1 PDF
    +-- simple-rl-design-v2.md    <- this document
    +-- simple-rl-design-v2.pdf   <- this document rendered
    +-- lora-verification-guide.md
    +-- lora-verification-guide.pdf
    +-- simple-rl-architecture.mmd
    +-- simple-rl-architecture.png
    +-- simple-rl-overview.mmd
    +-- simple-rl-overview.png
```

5 Deployment Reference (Updated)

5.1 Full Startup with GRPO Training

```
# 1. Start the policy server (oMLX -- single request at a time)
mlx_lm.server \
  --model mlx-community/Qwen2.5-0.5B-Instruct-4bit \
  --port 8080 \
  --api-key 1111

# 2. (Optional) Start the PRM server
mlx_lm.server \
  --model mlx-community/Qwen2.5-0.5B-Instruct-4bit \
  --port 8081 \
  --api-key 1111

# 3. Set GRPO training environment
export POLICY_MODEL_PATH=mlx-community/Qwen2.5-0.5B-Instruct-4bit
export MLX_TUNE_PYTHON=/Volumes/ExternalSSD/train/mlx-tune/.venv/bin/python3
export WANDB_API_KEY=<your-key>
export WANDB_PROJECT=terminal-rl-simple
export WANDB_ENTITY=bochuxt7-iot

# 4. Run training
cd /Volumes/ExternalSSD/train/OpenClaw-RL
python3 -m simple_rl.train_async \
  --dataset simple_rl/data/sample_tasks.jsonl \
  --max_concurrent 1 \
  --n_samples 4 \
  --rollout_batch_size 2 \
  --max_rounds 10
```

5.2 Environment Variable Quick Reference

```
# Policy server
export POLICY_URL=http://localhost:8080/v1
export POLICY_MODEL=Qwen2.5-0.5B-Instruct-4bit
export POLICY_API_KEY=1111
export POLICY_MAX_CONCURRENT=1       # must match oMLX capacity
export POLICY_MAX_RETRIES=3
export POLICY_RETRY_BACKOFF=2.0

# Episode parameters
export MAX_CONCURRENT=1              # keep <= POLICY_MAX_CONCURRENT
export N_SAMPLES_PER_PROMPT=4        # trajectories per task before GRPO
export ROLLOUT_BATCH_SIZE=2          # task groups per training round
export MAX_TURNS=20
```

```
# PRM (optional)
export PRM_ENABLE=1
export PRM_URL=http://localhost:8081/v1
export PRM_M=3

# GRPO training
export POLICY_MODEL_PATH=mlx-community/Qwen2.5-0.5B-Instruct-4bit
export GRPO_LORA_RANK=16             # do not change between rounds
export GRPO_LR=1e-6
export GRPO_NUM_GEN=4
export GRPO_BETA=0.04
export GRPO_MAX_STEPS=-1             # -1 = len(dataset)
export GRPO_OUTPUT_DIR=logs/grpo_adapters

# Observability
export LOG_DIR=logs
export WANDB_PROJECT=terminal-rl-simple
export WANDB_ENTITY=bochuxt7-iot
export WANDB_API_KEY=<key>
```

Document version 2 – generated 2026-03-29 from source: `simple_rl/` – OpenClaw-RL project.